

UNIVERSIDAD GALILEO
Postgrado en Diseño y Desarrollo de Software

INFRAESTRUCTURA EN LA NUBE
Proyecto de Curso

Sistema de Reservas y Disponibilidad de Parqueos

Entrega 1

INTEGRANTES

Ana Gabriela López Ortíz 18002988

Pablo Samuel Siquinajay Castro 22006477

Christian Javier Sosa Ruiz 22000102

Ciudad de Guatemala, 17 de mayo de 2026

ENTREGA 1

Resumen ejecutivo

Descripción del problema

Actualmente, muchas empresas cuentan con una cantidad limitada de espacios de parqueo en comparación con el número total de colaboradores. Con la implementación de modelos de trabajo híbrido (presencial y teletrabajo), la asistencia de empleados varía diariamente, lo que dificulta administrar de manera eficiente la disponibilidad de parqueos corporativos.

La falta de un sistema centralizado provoca conflictos de asignación, espacios desaprovechados y procesos manuales poco eficientes para gestionar reservas y disponibilidad.

Por ello, se propone desarrollar un Sistema de Reserva y Gestión de Parqueos Empresariales que permita a las empresas administrar de forma automatizada la disponibilidad de espacios para sus colaboradores. El sistema permitirá realizar reservas, visualizar disponibilidad en tiempo real, evitar conflictos de asignación y optimizar el uso de los parqueos disponibles, mejorando así la gestión interna y la experiencia de los colaboradores.

Actores

Actores Primarios

- **Usuario/Conductor:** Es la persona registrada en el sistema, la cual consulta disponibilidad, solicita y confirma reservas de espacios de parqueo.
- **Administrador:** Gestiona el inventario de espacios y consulta reportes de ocupación y asistencia, crea y edita características del parqueo.

Actores de Soporte

- **Servicio de autenticación:** Gestiona el registro e inicio de sesión de usuarios, controlando roles (Usuario o Administrador).
- **Servicio de almacenamiento en nube:** Persisten los comprobantes digitales generados tras cada reserva confirmada.
- **Servicio de mensajería asíncrona:** Ejecuta la liberación automática de espacios no confirmados tras 20 minutos y dispara las alertas de confirmación al usuario.
- **Servicio de notificaciones:** Envía alertas y confirmaciones de reserva al conductor (correo electrónico).

Casos de uso priorizados

1. Inicio de sesión

Como empleado, debo de acceder con un usuario previamente creado por el administrador e iniciar sesión para poder acceder al sistema de reservas.

Prioridad: Po

Criterio de éxito: El empleado inicia sesión con las credenciales asignadas por el administrador. El sistema emite un token de sesión válido con el rol correspondiente. Credenciales incorrectas retornan HTTP 401.

2. Consulta de disponibilidad en tiempo real

Como conductor, quiero consultar la disponibilidad de espacios filtrada por tipo de vehículo para saber si hay lugar antes de reservar.

Prioridad: Po

Criterio de éxito: El endpoint `GET /availability` retorna el estado de cada espacio (Disponible/Reservado/Ocupado) en tiempo real, respetando restricciones de rol del conductor.

3. Reserva de espacio

Como conductor, quiero reservar un espacio y recibir un comprobante digital para garantizar mi lugar.

Prioridad: Po

Criterio de éxito: El endpoint `POST /reservar` asigna el espacio atómicamente previniendo double-booking. El conductor recibe un comprobante QR en PDF por correo en menos de 30 segundos.

4. Gestor de usuarios para administrador

Como administrador, quiero gestionar los colaboradores registrados en el sistema, incluyendo crear, editar y desactivar cuentas y asignar roles. Controlar qué usuarios pueden reservar y qué tipo de espacios tienen permitido ocupar.

Prioridad: Po

Criterio de éxito: El administrador crea, edita y desactiva cuentas asignando roles (operativo, ejecutivo, visitante frecuente). Un usuario desactivado pierde acceso inmediatamente pero conserva su historial. Ningún colaborador puede modificar su propio rol ni el de terceros.

5. Gestor de parqueos para administrador

Como administrador, quiero configurar los espacios de parqueo disponibles, definiendo qué tipo de vehículo es permitido y qué roles de colaborador pueden ocuparlos, para que el sistema asigne reservas respetando las restricciones operativas de la empresa.

Prioridad: Po

Criterio de éxito: El administrador crea y edita espacios asignando identificador, tipo de vehículo y roles autorizados. Puede marcar espacios como inactivos por rango de fechas, quedando invisibles para reserva. Un colaborador con rol no autorizado no puede reservar un espacio restringido aunque esté disponible.

6. Liberación automática por no confirmación

Como sistema, quiero liberar automáticamente espacios no confirmados para optimizar la disponibilidad.

Prioridad: P1

Criterio de éxito: Si el conductor no confirma en 20 minutos, el espacio vuelve a Disponible de forma asíncrona y se notifica al conductor antes de la liberación.

7. Dashboard de ocupación para administrador

Como administrador, quiero visualizar un dashboard de ocupación para monitorear el estado del parqueo en tiempo real.

Prioridad: P2

Criterio de éxito: El dashboard presenta de forma gráfica el conteo de espacios en cada estado (Disponible, Reservado, Ocupado) actualizado en intervalos no mayores a 30 segundos. El administrador puede distinguir la ocupación por tipo de vehículo (moto/carro) sin necesidad de consultas adicionales.

8. Gestión de tarifas

Como administrador, quiero establecer precios diferenciados por tipo de vehículo.

Prioridad: P2

Criterio de éxito: El administrador define y actualiza tarifas para moto y carro. El cambio es inmediato y queda registrado con marca de tiempo para auditoría.

Funcionalidades específicas del proyecto:

1. Reserva con validación de placa registrada

El sistema solo permite reservar si la placa del vehículo está previamente registrada en el perfil del colaborador. No cualquier persona con cuenta puede reservar — tiene que haber un vehículo vinculado y aprobado. Un colaborador puede tener más de una placa (ej. carro y moto), pero solo puede tener una reserva activa por día.

2. Asignación de tipo de espacio por perfil de colaborador

El Admin puede categorizar colaboradores: ejecutivo, operativo, visitante frecuente. Cada categoría tiene acceso a un pool de espacios distinto. Un colaborador operativo no puede reservar un espacio marcado como ejecutivo aunque esté disponible. Esto refleja jerarquías reales que las empresas ya manejan manualmente hoy.

3. Historial de asistencia vehicular por colaborador

Dado que cada reserva está vinculada a un colaborador identificado, el sistema genera automáticamente un registro de qué días llegó en vehículo, a qué hora confirmó su llegada y cuánto tiempo ocupó el espacio. El admin puede exportar esto. Es un diferenciador fuerte porque las empresas hoy no tienen visibilidad de esto.

4. Liberación anticipada voluntaria

El colaborador puede liberar su espacio antes de la hora de salida desde la app (ej. salió temprano). Esto actualiza disponibilidad en tiempo real y permite que otro colaborador tome ese espacio el mismo día. Es distinto al timeout automático de 20 minutos — esta es una acción explícita del usuario que beneficia al resto.

5. Política de cancelación con penalización configurable

Si un colaborador cancela una reserva menos de N horas antes (configurable por el admin), el sistema lo registra como "cancelación tardía". Acumular X cancelaciones tardías en un mes puede bloquear temporalmente la capacidad de reservar. Esto desincentiva el comportamiento de reservar por precaución sin intención real de usar el espacio.

Mockups del FE

Se diseñaron 7 pantallas low-fi que cubren los casos de uso priorizados (Po, P1 y P2). Los wireframes completos se encuentran en el Anexo A.

Mapeo a conceptos del curso

Esta sección vincula las necesidades operativas de nuestro sistema de parqueos con el stack tecnológico del curso, vincula los componentes con las funcionalidades específicas que nuestro *Sistema de Reservas y Disponibilidad de Parqueos* va a hacer.

Componente del curso	Cómo lo ejercita su proyecto
Cómputo (API)	- Endpoint <code>GET/availability</code> consulta disponibilidad de parqueo filtrando por tipo de vehículo (moto, carro) del lado del cliente. - Endpoint <code>POST/reservar</code> ejecuta la lógica de validación de disponibilidad y asigna espacios en ese mismo instante.
Base de datos	Inventario de espacios con estados <i>Libre/Ocupado</i> y almacenamiento de datos de usuario registrados(perfil de datos) + sus placas vehiculares.

Almacenamiento de archivos	• Comprobantes digitales(en PDF/PNG) de finalización de reserva del parqueo. • Hosting de archivos estáticos; como imágenes de referencia de las entradas de los parqueos. Ayuda visual para el conductor.
Procesamiento asíncrono	• Si el usuario no confirma digitalmente su ingreso tras 20 minutos de la hora pactada, un <i>Worker</i> asíncrono procesa la cancelación, libera el cupo en la base de datos y lo marca el espacio de “Reservado” a “Disponible”.
Red	• El sistema expone únicamente el API Gateway a internet público vía HTTPS. Los servicios internos (base de datos, worker de liberación asíncrona) residen en subredes privadas sin acceso directo desde el exterior.
Seguridad	• Roles de IAM ; El "<i>Admin</i>" solo puede ver reportes y gestionar; el "<i>User</i>" solo genera reservas. • Cifrado de datos sensibles (ej. <code>license_plate_number_user</code> y <code>user_phone_number</code>).
Observabilidad	• <i>Alarma de saturación: notificación automática si la disponibilidad de espacios llega a cero</i>

Scope (in/out)

Declaración explícita de qué hará el sistema y qué queda fuera del diseño.

- **In:**
 - Gestión de identidad(Auth); Para el registro de usuarios al sistema. Mejora el control de usuarios que ingresan al sistema y los permisos que tiene(User/Admin)
 - Búsqueda de espacios disponibles; Muestra los espacios disponibles de parqueo de la empresa.
 - Gestión automática de reservas; Mediante una reserva de espacio en tiempo real que asegure atomicidad de espacio(ej. para que 2 o más personas no reserven el mismo espacio al mismo tiempo). El sistema rechaza a uno y rechaza al otro por orden de reserva.
 - Liberación asíncrona; Cancelación automática por no confirmación el día programado (asíncrono), con una espera máxima de 20 minutos y alertas de confirmación, entonces, liberar espacio antes ocupado.
 - Generación de comprobantes de confirmación de reservas; Generación de comprobante único digital e inalterable para el acceso. El sistema genera un QR en PDF que se guarda en el almacenamiento de la nube.
 - Dashboard interactivos de Disponibilidad/Ocupación;

- Un cliente podrá
 - Consultar disponibles(tendrá mensajes en la UI de “Ocupado”/”Disponible”),
 - Solicitar reserva, confirmar reserva.
 - Marcar en la app un botón de “Ocupar Parqueo”, y el estado en la BD cambia a "Ocupado"
- Para el Admin, podrá ver
 - En su inventario(mediante algún tipo de gráfico) los espacios disponibles, espacios reservados y actualmente ocupados.
 - Establecer el precio del parqueo por moto y carro.
 - Que usuarios han reservado y que días (brinda control de asistencia)
 - Respecto a la disponibilidad del parqueo, se usarán cantidad de parqueos disponibles(por motivos prácticos y sencillez) para ubicar qué número de parqueo es el que se encuentra como “Ocupado”/”Disponible”.
- **Out:**
 - Integración física con hardware.
 - Procesamiento de pagos reales; Por tema de complejidad, y costos.
 - **App Móvil Nativa**

Preguntas abiertas E1

¿Qué mecanismo concreto garantiza la atomicidad en la asignación de espacios? Se menciona que "el sistema rechaza a uno por orden de reserva", pero no se especifica si se usará locking optimista, pesimista, transacciones ACID, o algún mecanismo de cola (queue).

¿Cómo se maneja el race condition a nivel de base de datos cuando dos requests llegan en el mismo milisegundo?

¿Qué tecnología ejecuta la liberación automática a los 20 minutos? No se define si es un cron job, un mensaje en una cola (SQS, RabbitMQ), un scheduled function (Cloud Scheduler, Lambda EventBridge), o un timer en base de datos. Cada opción tiene implicaciones de confiabilidad distintas.

¿Cómo integraremos una función más eficaz cuando el cliente llegue “verdaderamente” al parqueo más que solo un clic de “Ocupar parqueo”? ¿Un IoT como sensor de parqueo sería suficiente?

Anexos

Anexo A: Mockups

Pantalla 1 - Registro e inicio de sesión: Esta pantalla cubre el acceso al sistema. Se presentan dos vistas: el formulario de inicio de sesión para usuarios existentes y el formulario de registro para nuevos usuarios.

The mockup shows two side-by-side white cards on a light beige background. The left card is for login, featuring a 'P' icon, the title 'ParkReserva', a login button, and fields for email and password. The right card is for registration, titled 'Crear cuenta', with fields for name, surname, email, vehicle type, license plate, and password, followed by a registration button.

ParkReserva
Iniciar sesión

Correo electrónico

Contraseña

Ingresar

¿No tienes cuenta? [Regístrate](#)

Crear cuenta
Completa tus datos

Nombre

Apellido

Correo electrónico

Tipo de vehículo

Placa

Contraseña

Crear cuenta

Pantalla 2 - Consulta de disponibilidad en tiempo real: Esta pantalla permite al conductor visualizar el estado actual de los espacios de parqueo. Se presenta una cuadrícula con cada espacio identificado por número, mostrando su estado mediante código de color: verde (Disponible), rojo (Ocupado) y amarillo (Reservado). El conductor puede filtrar por tipo de vehículo (moto/carro) antes de proceder a reservar.

The mockup displays a grid of parking spaces with their status (Available, Reserved, Occupied) and a summary of counts at the bottom. A filter for vehicle type (Car/Moto) is located at the top right.

Disponibilidad de espacios
Actualizado hace 3 segundos

■ Disponible

■ Reservado

■ Ocupado

A-01	A-02	A-03	A-04	A-05
B-01	B-02	B-03	B-04	B-05

5

Disponibles

2

Reservados

3

Ocupados

Reservar espacio

Pantalla 3 - Flujo de reserva: Guía al conductor a través del proceso de reserva de un espacio seleccionado. Se muestran los detalles del espacio elegido, los datos del vehículo registrado y la hora estimada de llegada. El sistema valida la disponibilidad de forma atómica al momento de confirmar, previniendo el double-booking. Al completarse, se genera el comprobante QR.

Confirmar reserva
Revisa los detalles antes de continuar

Espacio seleccionado

 **Espacio A-02**
Zona A - Nivel 1 Disponibile

Datos del vehículo

Tipo: **Carro** Placa: **P-123ABC**

Hora estimada de llegada
08:30 AM 

El espacio se liberará automáticamente si no es confirmado en 20 minutos.

Cancelar **Confirmar reserva**

Pantalla 4 - Comprobante de reserva con código QR: Una vez confirmada la reserva, el sistema genera y presenta al conductor un comprobante digital único con código QR. Este comprobante queda disponible para descarga inmediata en formato PDF y es enviado al correo registrado del conductor. El código QR es inalterable y sirve como credencial de acceso al parqueo.



Reserva confirmada
Tu comprobante ha sido generado

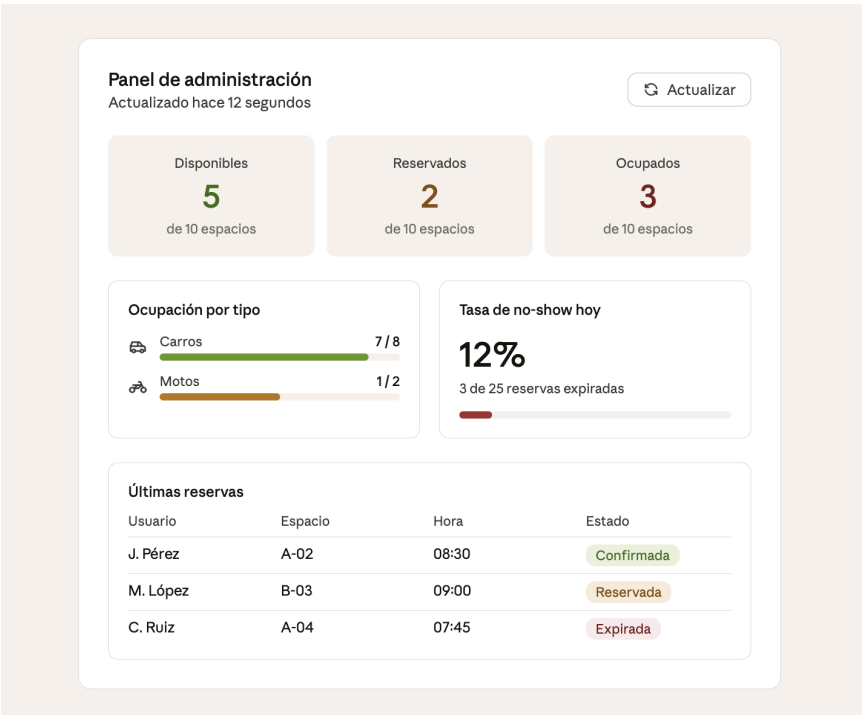
No. de reserva	#RES-20260515-0042
Espacio asignado	A-02 - Zona A
Placa	P-123ABC
Hora de llegada	08:30 AM - 15 may 2026



Presenta este QR al ingresar al parqueo

 Descargar PDF  Enviar correo

Pantalla 5 - Dashboard del administrador: Esta pantalla provee al administrador una visión general en tiempo real del estado del parqueo. Se presentan métricas de ocupación mediante gráficos y contadores por estado, diferenciados por tipo de vehículo.



Pantalla 6 - Gestión de precios por tipo de vehículo: Esta pantalla permite al administrador configurar y actualizar las tarifas diferenciadas por tipo de vehículo. Cada cambio queda registrado en el historial, para fines de auditoría.

Gestión de tarifas
Configura el precio por tipo de vehículo

Carro
Tarifa por hora (Q)
Q 10.00

Moto
Tarifa por hora (Q)
Q 5.00

Historial de cambios

Fecha	Tipo	Tarifa
15 may, 08:00	Carro	Q 10.00
10 may, 14:30	Moto	Q 5.00
01 may, 09:15	Carro	Q 8.00

Cancelar

Guardar cambios

Pantalla 7- Historial de reservas por usuario y fecha: Esta pantalla permite al administrador consultar y auditar el registro completo de reservas del sistema. La tabla es filtrable por usuario y rango de fechas, mostrando para cada entrada el espacio asignado, tipo de vehículo, estado final y marca de tiempo. Los registros son inmutables; ningún rol puede eliminarlos.

Historial de reservas
Registro inmutable de todas las reservas

Exportar

Usuario: Todos los usuarios | Fecha inicio: 01 may 2026 | Fecha fin: 15 may 2026

Usuario	Espacio	Vehículo	Fecha / Hora	Estado
Juan Pérez	A-02	Carro	15 may · 08:30	Confirmada
María López	B-03	Moto	15 may · 09:00	Reservada
Carlos Ruiz	A-04	Carro	14 may · 07:45	Expirada
Ana González	B-01	Moto	13 may · 10:15	Cancelada
Juan Pérez	A-01	Carro	12 may · 08:00	Confirmada

Mostrando 5 de 42 registros

Anterior | Siguiente

Anexo B — Anexo de IA

1. Prompt #1

"Actúa como un Arquitecto de Soluciones Cloud. Tengo un sistema de reserva de parqueos en Guatemala para un proyecto. Ayúdame a:

- 1. Redactar el cierre del Resumen Ejecutivo enfocándose en qué procesos se van a automatizar (evitar double-booking, liberación de espacios).*
- 2. Generar ejemplos específicos para una tabla que mapee las funcionalidades del sistema con los 7 componentes obligatorios del curso (Cómputo, DB, Almacenamiento, Red, Asíncrono, Seguridad, Observabilidad).*
- 3. Asegúrate de que los ejemplos no sean genéricos (no digas solo 'hacer login') sino que mencionen procesos reales de un parqueo inteligente."*

Nuestra reflexión

- **Qué aceptamos:** La estructura de la tabla y los ejemplos de procesos asíncronos, ya que es un "trade-off" interesante para el negocio.
- **Qué editamos:** Editamos ejemplos de procesos asíncronos, seguridad, red, observabilidad, Almacenamiento de archivos.
- **Qué descartamos:** Sugerencias de la IA sobre algunas funciones de red, como aislamiento de la base de datos y la lógica de negocio.

2. Prompt #2

"Actúa como Product Owner y genera una lista de Scope In/Out para un sistema de parqueos inteligente enfocado en un curso de Infraestructura Cloud, priorizando funcionalidades que usen bases de datos y colas de mensajes."

Reflexión del equipo

- **Qué aceptamos:** La estructura de "In/Out" sugerida por la IA, especialmente la idea de dejar fuera la integración física (sensores) y el desarrollo para una app móvil para no complicar el alcance técnico del curso.
- **Qué editamos:** Todas las sugerencias, ajustándose a un desarrollo más práctico y sencillo.
- **Qué descartamos:** La IA sugirió incluir "Machine Learning para predicción de demanda"; lo movimos a *Out-of-Scope* inmediatamente para mantener el proyecto ejecutable en 3 meses.

ENTREGA 2

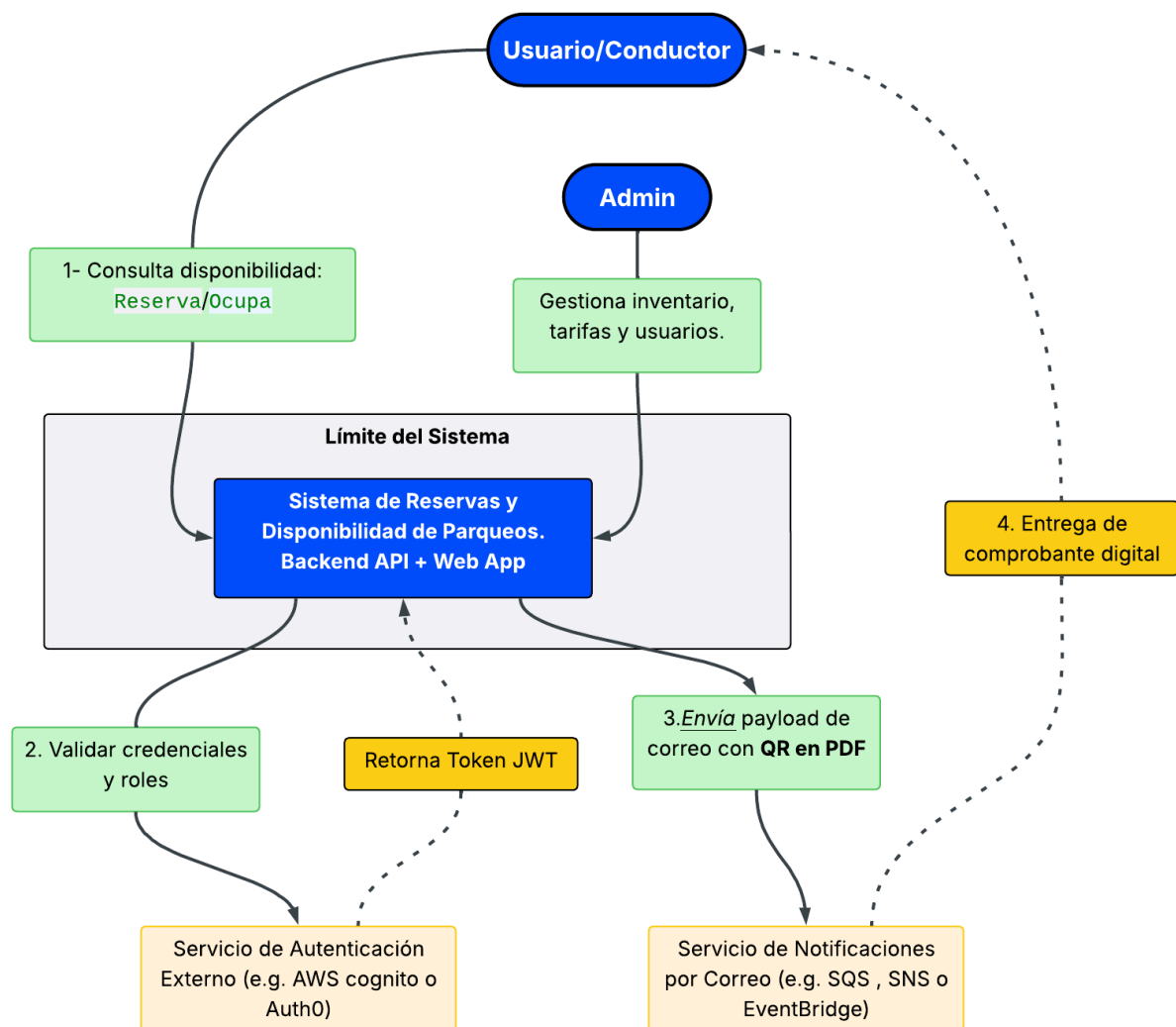
Resumen de cambios

- **Definición de la Estrategia de Concurrencia (Atomicidad):**

Se resolvió la duda de la *Entrega 1* sobre el mecanismo para evitar el *double-booking*. Se descartó la necesidad de implementar colas de mensajes bloqueantes o bloqueos pesimistas a nivel de aplicación; en su lugar, se optó por delegar la atomicidad a la base de datos mediante **índices únicos parciales** en PostgreSQL, simplificando la capa de cómputo.

Diagrama de contexto

El siguiente diagrama delimita las fronteras del Sistema de Reserva y Gestión de Parques Empresariales, también, definiendo los actores externos y sistemas de terceros.



Descripción de las Entidades del Diagrama

A. Límite del Sistema (System Boundary)

- **Sistema de Reservas y Disponibilidad de Parqueos:** Comprende todo el software desarrollado por el equipo (interfaces web de usuario/administrador y el backend API de funciones). Se encarga de procesar la lógica de negocio, calcular la disponibilidad en tiempo real, derivar estados vehiculares y gobernar la persistencia de datos relacionales y punteros de archivos.

B. Actores Externos (Usuarios Primarios)

- **Usuario/Conductor:** Interactúa con el sistema para consultar la disponibilidad de espacios en tiempo real, enviar solicitudes de reserva, registrar voluntariamente la ocupación o liberación anticipada y recibir sus comprobantes digitales.
- **Administrador (Personal de operaciones de la empresa):** Interactúa con el sistema para dar de alta o baja cuentas de colaboradores, configurar restricciones vehiculares por perfil(solo moto o carro) y auditar el historial de asistencia vehicular de la corporación.

C. Sistemas Externos de Terceros (Sistemas de Soporte)

- **Servicio de Autenticación Externo (e.g., AWS Cognito / Auth0):** Sistema gestionado externo encargado de validar las credenciales de acceso al sistema. Le quita la responsabilidad al sistema de almacenar contraseñas, retornando tokens criptográficos (JWT) que contienen el rol (`system_role`) del usuario para la posterior autorización en el API.
- **Servicio de Notificaciones (e.g., AWS SQS/SNS/Event Bridge):** Plataforma externa de entrega de correos electrónicos. Recibe las instrucciones del sistema transaccional de parqueos junto con el archivo PDF/PNG del comprobante generado, y se encarga de enviarlo directamente al buzón del conductor final.

Decisión de cómputo

Servicio elegido: AWS ECS Fargate

El backend del sistema se desplegará como un contenedor Docker gestionado por ECS Fargate. Fargate es la modalidad serverless de contenedores en AWS: el equipo define la task definition (imagen, CPU, memoria, puertos) y AWS gestiona el servidor, el sistema operativo y los parches. No se administra ninguna instancia EC2 directamente.

Por qué Fargate sobre las alternativas

AWS Lambda fue descartado porque el sistema expone una API REST con múltiples endpoints que necesitan estar disponibles de forma continua. Lambda está diseñado para ejecuciones cortas y puntuales y no es el modelo adecuado para un servidor HTTP

persistente con conexiones a base de datos que debe responder en tiempo real a consultas de disponibilidad cada 30 segundos.

EC2 con Auto Scaling fue descartado porque requiere administrar el servidor subyacente: actualizaciones del sistema operativo, parches de seguridad, configuración del runtime y gestión del ciclo de vida de las instancias. Para un equipo de tres personas en un proyecto de curso, ese overhead operativo no agrega valor al diseño del sistema.

Trade-offs

1. Disponibilidad continua vs. costo en baja demanda

Fargate mantiene el contenedor corriendo aunque no haya tráfico, lo que genera costo incluso en horarios de baja actividad (noches, fines de semana). Lambda cobraría solo por invocación y resultaría más económico para un sistema con uso muy intermitente. Para este sistema, donde el dashboard del administrador se refresca cada 30 segundos durante el horario laboral y las reservas ocurren en ventanas predecibles de la mañana, la disponibilidad continua está justificada: el costo de una respuesta lenta o un cold start en el momento pico de reservas (07:00–09:00) supera el ahorro nocturno.

2. Sin servidor que administrar vs. menor granularidad de control.

Al delegar la gestión del servidor a AWS, el equipo gana velocidad operativa pero pierde control sobre la configuración del sistema operativo, la red a nivel de host y la capacidad de instalar dependencias del sistema directamente. Para el backend de este proyecto, una API REST con dependencias definidas y empaquetadas en un contenedor, ese nivel de control no es necesario. Si en el futuro el sistema requiriera acceso a hardware especializado o configuraciones de red avanzadas a nivel de instancia, EC2 sería la opción correcta.

Desventaja reconocida

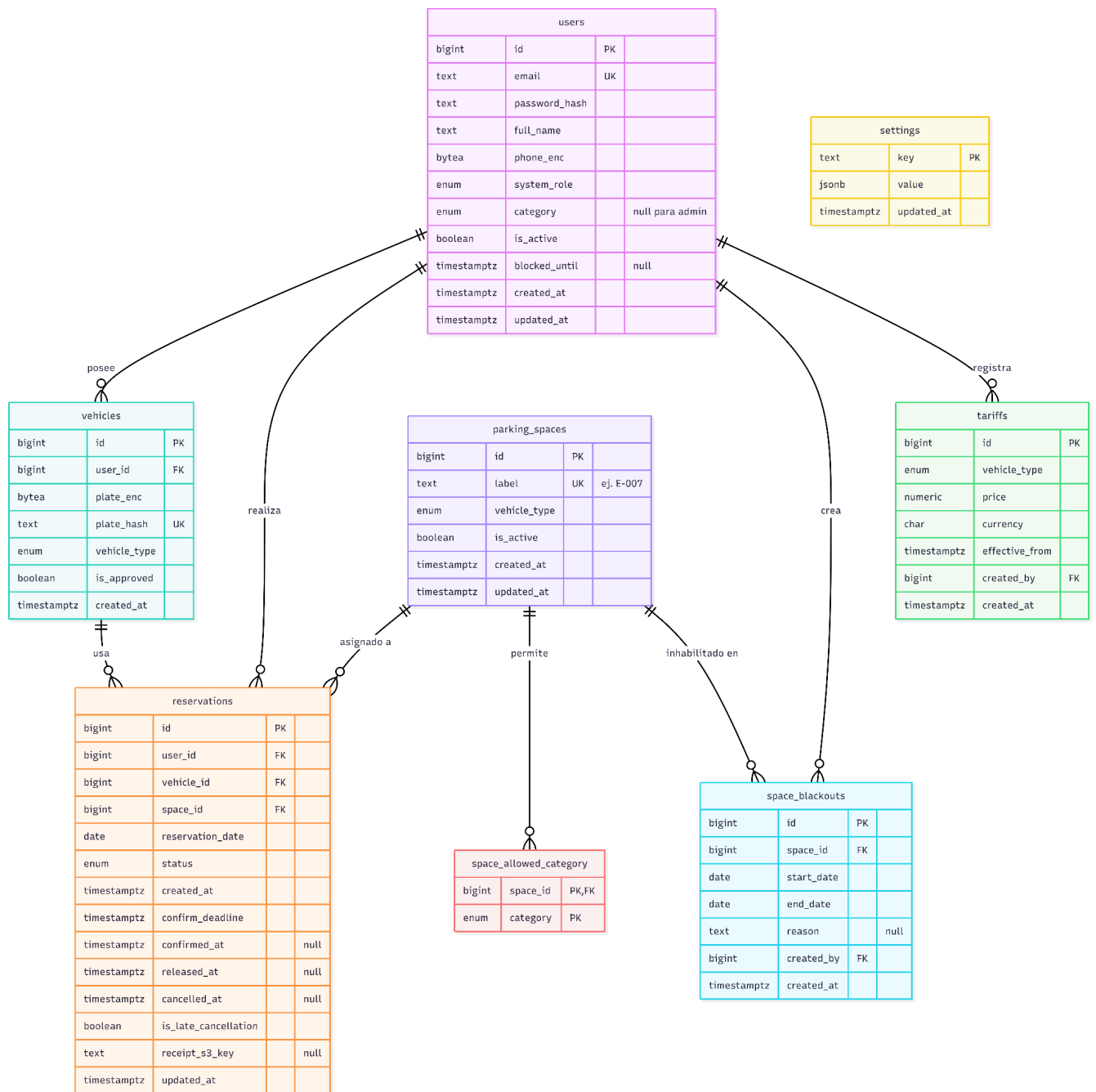
Fargate tiene una latencia de arranque de 20 a 30 segundos cuando escala desde cero tareas. Si el servicio se detiene por inactividad y recibe una solicitud en ese instante, la primera respuesta se verá afectada. Para el MVP se acepta este riesgo manteniendo al menos una tarea corriendo en todo momento, lo que elimina el cold start a costa de no escalar a cero. En producción, esto se mitigaría con una política de Auto Scaling que nunca baje de una tarea mínima.

Modelo de datos

El dominio se modela como un esquema relacional sobre PostgreSQL 16 (RDS), en modo single-company (un despliegue para una empresa, sin tabla de tenants). Se eligió un modelo relacional porque el negocio gira en torno a invariantes transaccionales (no permitir doble reserva, una reserva activa por día) y la integridad referencial entre colaboradores, vehículos, espacios y reservas, estas son propiedades que una base relacional garantiza de forma nativa.

Entidades principales

Entidad	Rol en el dominio
users	Colaboradores y administradores en una sola tabla, diferenciados por <code>system_role</code> .
vehicles	Vehículos del colaborador (1:N). Reservar exige una placa registrada y aprobada.
parking_spaces	Inventario de espacios; cada uno con un tipo de vehículo admitido.
space_allowed_category	Relación M:N que define qué categorías de colaborador pueden usar cada espacio.
space_blackouts	Inhabilitación de un espacio por rango de fechas.
reservations	Núcleo del modelo: sostiene disponibilidad, atomicidad, liberación, asistencia y el comprobante.
tariffs	Historial de precios append-only (auditoría); el precio vigente es la fila más reciente por tipo.
settings	Política configurable por el admin (ventana de cancelación, máximo de cancelaciones tardías).



Tres principios sostienen la coherencia del modelo

1. El estado del espacio (**Disponible/Reservado/Ocupado**) se *deriva*, no se **almacena**. Se calcula por (**espacio, fecha**) a partir de la reserva activa. Una columna **estado** fija no puede expresar "ocupado hoy, libre mañana" y se desincronizaría con las reservas.
2. **Atomicidad por índice único parcial**, no por bloqueos de aplicación (ver patrón de acceso 2).
3. **Dos ejes de "rol" separados**: **system_role** (admin/driver, para permisos) y **category** (ejecutivo/operativo/visitante_frecuente, para el pool de espacios accesible). No se fusionan.

Patrones de acceso principales

#	Patrón de acceso	Tipo	Frecuencia	Estructura que lo soporta
1	Consultar disponibilidad (GET /availability), filtrada por tipo de vehículo y categoría del conductor	Lectura	Muy alta	LEFT JOIN parking_spaces ↔ reservations del día activo; índice (reservation_date, status); exclusión de inactivos y space_blackouts.
2	Crear reserva (POST /reservar), atómica	Escritura (alta contención)	Alta	Índices únicos parciales (space_id, reservation_date) y (user_id, reservation_date) con WHERE status IN ('reservada','ocupada').
3	Confirmar llegada / liberar (ocupar, salida, liberación anticipada)	Escritura	Media	UPDATE reservations de status + marcas de tiempo (confirmed_at, released_at).
4	Liberación automática a los 20 min (worker P1)	Lectura + escritura batch	Periódica	Índice (status, confirm_deadline) para escanear reservas por expirar.
5	Historial de asistencia por colaborador (export del admin)	Lectura	Baja	Índice (user_id, reservation_date); campos confirmed_at/released_at.
6	Conteo de cancelaciones tardías del mes (penalización Fn5)	Lectura	Baja	is_late_cancellation + índice (user_id, reservation_date).
7	Precio vigente / historial de tarifas	Lectura	Baja	tariffs append-only + vista current_tariffs (DISTINCT ON (vehicle_type)).
8	Gestión (CRUD) de usuarios y espacios	Escritura	Baja	Claves primarias e índices únicos (email, label, plate_hash).

El patrón #1 es el más frecuente (la pantalla de disponibilidad se refresca cada ≤ 30 s) y por eso se optimiza con un índice dedicado y un conjunto de datos pequeño (decenas de espacios). El patrón #2 es el más crítico en correctitud: la **atomicidad la garantiza la base de datos**. Dos reservas simultáneas para el mismo espacio/día ejecutan **INSERT** concurrentes; el índice único parcial deja pasar solo uno y el otro recibe una violación de unicidad que el API traduce a **HTTP 409**. Es ACID, sin bloqueo pesimista ni cola de mensajes, responde la pregunta de E1 sobre cómo se evita el double-booking.

Qué va en base de datos vs. almacenamiento de objetos

Dato	Dónde	Justificación
Usuarios, vehículos, espacios, reservas, tarifas, settings	PostgreSQL (RDS)	Datos estructurados y relacionales, registros pequeños, requieren transacciones (reserva atómica), integridad referencial (FKs) y consultas indexadas (disponibilidad, historial).
Comprobantes de reserva (QR en PDF/PNG)	S3 (object storage)	Blobs binarios, inmutables (write-once), de tamaño medio, que se sirven por descarga/URL prefirmada. No requieren consultas relacionales; almacenarlos en BD inflaría el tamaño y degradaría backups.
Imágenes estáticas de referencia (entradas del parqueo)	S3	Contenido estático servido directamente; no es dato transaccional.
Puntero al comprobante (<code>reservations.receipt_s3_key</code>)	PostgreSQL	La BD guarda solo la clave/URL del objeto en S3, no el binario. Mantiene la fila ligera y enlaza el dato relacional con el objeto.

Decisión para MVP: no se introduce una capa de caché dedicada (ej. Redis/ElastiCache). La consulta de disponibilidad (patrón #1) opera sobre un conjunto pequeño (decenas de espacios) con un índice apropiado, por lo que PostgreSQL cumple holgadamente los SLAs (< 2 s de respuesta, reflejo del cambio en ≤ 30 s) sin caché. Añadir una caché ahora introduciría un problema de **invalidación y consistencia** (datos desactualizados) que contradice el requisito de tiempo real, a cambio de un beneficio de rendimiento que aún no se necesita.

Preguntas abiertas E2

1. ¿Mantener enums para **vehicle_type**/categorías o migrar a tablas de catálogo si el negocio necesita agregarlos sin desplegar cambios de esquema?
2. ¿Cuándo conviene **particionar reservations por fecha** al crecer el historial, y qué política de retención/archivado aplica?
3. Dado que la base de datos PostgreSQL (RDS) y el backend se alojarán en subredes privadas (acorde a E1 en *Componentes del Curso*), ¿cuál será el mecanismo de red óptimo para que nuestro cómputo se comunique de forma segura con Amazon S3 para subir los PDFs de reservas?
 - a. ¿Utilizaremos un *Gateway VPC Endpoint* para evitar que el tráfico viaje por la internet pública o abriremos rutas de NAT Gateway?
4. Para cuando **Disponibilidad = 0** ¿Es más eficiente calcular el estado en tiempo real con *Lambda* periódica que publique una métrica customizada en *CloudWatch*, o deberíamos emitir **eventos** directamente desde la BD en cada mutación de reserva?

Anexo IA

1. Lo que le pedimos a la IA

- *"Actúa como un arquitecto de datos en la nube. Ayúdanos a estructurar una estrategia de persistencia para un sistema de reservas de parqueos corporativos de tamaño mediano (decenas de espacios). Compáranos el uso de un enfoque puramente relacional (PostgreSQL) contra uno NoSQL (DynamoDB), considerando la regla de negocio crítica: evitar double-booking y garantizar una sola reserva por día por usuario."*

2. Análisis y Reflexión del Equipo

- **Qué aceptamos:** En primera instancia la IA nos recomendó 100% una Base de datos relacional (ej. PostgreSQL). Evaluamos nuestra lógica de negocio, como por ejemplo *"permitir solo una reserva activa por usuario por día"* o *"validar si el espacio pertenece a la categoría asignada al colaborador"*. Estas requieren un aseguramiento en la integridad y consistencia en datos, además de manejar consistencia en datos, algo que con los principios ACID de una BD relacional nos queda excelente.
- **Qué editamos:**
- **Qué descartamos:** La IA sugirió implementar una capa de almacenamiento en caché en memoria distribuida usando *Redis (Amazon ElastiCache)* para acelerar el endpoint de disponibilidad de espacios (**GET /availability**), bajo el pretexto de que la lectura se actualiza cada 30 segundos. El equipo descartó la propuesta de inmediato para el MVP, debido a que el volumen de datos de un parqueo mediano no justifica la complejidad técnica extra de controlar la invalidación de la caché, pudiendo resolverse eficientemente con índices nativos en PostgreSQL sin generar

inconsistencias de datos en tiempo real. Añadir una caché ahora introduciría un problema de **invalidación y consistencia** (datos desactualizados) que **contradice el requisito de tiempo real**, a cambio de un beneficio de rendimiento que aún no se necesita.